SAMSUNG SW AI ACADEMY FOR YOUTH

對外秘

프로젝트 명세서

AI 기반 도서 분석과 창작 지원 통합 솔루션 서비스

(Python)

"관계형 DB 설계"

6 회차

對外秘

목차

1. 관계형 DB 설계 3

1.1 목표 3

1.2 준비사항 3

1.3 작업 순서 3

1.4 요구사항 4

1.5 참고자료 35

1.6 프로젝트 결과 35

對外秘

1. 관계형 DB 설계

1.1 목표

이번 '프로젝트 기획과 협업' 프로젝트의 목표는 다음과 같습니다.

- 데이터를 생성, 조회, 수정, 삭제할 수 있는 Web application을 제작한다.
- Django Model과 ORM에 대하여 이해한다.
- Django Form에 대하여 이해한다.
- Django Authentication System에 대하여 이해한다.
- Many to one relationship(N:1)에 대하여 이해한다.
- Many to many relationship(M:N)에 대하여 이해한다.

1.2 준비사항

1) 제공 파일

- 없음

2) 개발언어 및 툴

- Python 3.11
- HTML, CSS
- Visual Studio Code
- Google Chrome

3) 필수 라이브러리 / 오픈소스

- Django 5.2
- Bootstrap 5.3

1.3 작업 순서

1) 팀원과 같이 요구사항을 확인하고, GitLab에 프로젝트를 생성한다.

- 프로젝트 이름은 06-pjt로 지정한다.
- 각 반 담당 강사님을 Maintainer 로 설정한다.

2) 팀원과 합의하여 협업 방식을 결정하고, 요구사항을 구현한다.

3) 작성한 코드들을 정리하고, README 를 작성한다.

- .gitignore 파일을 활용하여 불필요한 파일 및 폴더는 제출하지 않는다.

4) README 작성이 완료되면 심화 학습을 진행한다.

5) 제출 기한에 맞춰 모든 산출물이 GitLab 에 업로드 될 수 있도록 한다.

1.4 요구사항

본 프로젝트는 AI 기반 도서 분석과 창작 지원 통합 솔루션 서비스 구축을 목표로 합니다. 다양한 장르와 형태의 도서 데이터를 수집 및 관리하고, 이를 기반으로 AI 기반 도서 분석 기능을 설계 및 구현합니다.

작가들에게 아이디어 발상 지원, 플롯 구성 제안, 문장 교정 및 스타일 개선 등 창작 전반에 걸친 다양한 AI 기반 창작 지원 기능을 제공합니다. 또한 도서에 대한 사용자 리뷰 및 감상평 공유 커뮤니티 기능을 제공하여 사용자들이 활발하게 소통하고 정보를 교환할 수 있는 기능을 제공합니다.

사용자는 자신이 읽은 도서를 평가하고, 다른 사용자의 리뷰를 참고하여 다음 도서를 선택하는 데 도움을 받을 수 있습니다. 더 나아가, 관심 도서 목록을 맞춤형으로 구성하는 등 다양한 편의 기능을 제공합니다. 팀원과 상의하여 아래 요구사항을 만족할 수 있도록 요구사항 명세서를 작성 및 구현해봅시다.

도서를 읽은 후 감상을 공유하는 기능을 만드는 단계입니다. 도서 데이터의 생성, 조회, 수정, 삭제가 가능하며, 해당 도서를 읽은 후 감상을 게시글의 형태로 생성, 조회, 수정, 삭제가 가능한 애플리케이션을 완성해봅시다.

■ 요구사항 예시 (참고용)

- 아래의 내용을 참고하여 추가적인 아이디어에 대해 요구사항을 추가 또는 수정하여 기능을 구현합니다. 단, **필수 기능은 반드시 구현해야 하며 수정할 수 없습니다.**

기능적 요구사항

對外秘

번호	분류	요구사항명	요구사항 상세	우선순위
F601	프로젝트	프로젝트 구성	도서 커뮤니티 서비스 구현을 위한 Django 프로젝트 및 앱 생성	필수
F602	accounts	Model Class	사용자 데이터를 데이터베이스에 저장할 수 있도록 AbstractUser를 상속받는 Django Model 클래스 구현	필수
F603	accounts	Form Class	사용자가 입력한 사용자 데이터 검증을 위해 적절한 Form 클래스 활용 및 구현	필수
F604	books	Model Class	도서 데이터를 데이터베이스에 저장할 수 있도록 Django Model 클래스 구현	필수
F605	books	Model Class	도서에 대한 쓰레드 데이터를 데이터베이스에 저장할 수 있도록 Django Model 클래스 구현	필수
F606	books	Form Class	사용자가 입력한 데이터 검증을 위한 Django Form 클래스 구현	필수
F607	accounts view 함수	login	로그인 UI를 출력하고, 유효성 검증 및 세션 데이터를 저장하는 view 함수 구현	필수
F608	accounts view 함수	logout	현재 인증된 사용자의 세션 관련 데이터를 삭제하는 함수 구현	필수
F609	accounts view 함수	signup	회원가입 UI를 출력하고, 유효성 검증 및 회원 데이터를 저장하는 view 함수 구현	필수
F610	accounts view 함수	profile	개별 사용자의 정보를 확인할 수 있는 페이지 구현	필수
F611	accounts view 함수	follow	한 사용자가 다른 사용자를 팔로우 하는 기능	필수
F612	accounts view 함수	update	회원정보수정 UI를 출력하고, 유효성 검증 및 회원 데이터를 수정하는 view 함수 구현	필수
F613	accounts view 함수	delete	현재 인증된 사용자의 데이터를 삭제하는 함수 구현	필수
F614	accounts view 함수	change_password	비밀번호 변경 UI를 출력하고, 유효성 검증 및 비밀번호를 변경하는 view 함수 구현	필수
F615	books view 함수	index	전체 도서 데이터를 조회하는 view 함수 구현	필수

F616	books view 함수	create	도서 데이터를 입력할 수 있는 UI를 제공하고, 데이터에 따라 도서 데이터를 작성자 정보와 함께 저장하는 view 함수 구현	필수
F617	books view 함수	detail	단일 도서 데이터를 조회하는 view 함수 구현	필수
F618	books view 함수	update	도서 데이터를 입력할 수 있는 UI를 제공하고, 선택한 도서 데이터를 수정하는 view 함수 구현	필수
F619	books view 함수	delete	단일 도서 데이터를 삭제하는 view 함수 구현	필수
F620	books view 함수	thread_create	쓰레드 데이터를 입력할 수 있는 UI를 제공하고, 데이터에 따라 쓰레드 데이터를 작성자 정보와 함께 저장하는 view 함수 구현	필수
F621	books view 함수	thread_detail	쓰레드 데이터를 조회하는 view 함수 구현	필수
F622	books view 함수	thread_update	쓰레드 데이터를 입력할 수 있는 UI를 제공하고, 데이터에 따라 쓰레드 데이터를 수정하는 view 함수 구현	필수
F623	books view 함수	thread_delete	쓰레드 데이터를 삭제하는 view 함수 구현	필수
F624	books view 함수	like	쓰레드 데이터를 좋아요 하거나 취소하는 view 함수 구현	필수
F625	AI 활용	정보 생성 및 저장	생성형 AI API를 활용하여 작자 정보 검색 및 생성	심화
F626	AI 활용	커버 이미지	쓰레드 작성 시 커버 이미지를 생성형 AI로 생성하기	심화
F627	AI 활용	쓰레드 댓글	쓰레드에 대한 댓글 생성 및 삭제 기능 구현	심화
F628	AI 활용	카테고리	사용자의 선호 장르와 도서의 장르를 나타내기 위한 카테고리 Django Model 구현	심화

비기능적 요구사항				
번호	**분류**	**요구사항명**	**요구사항 상세**	**우선순위**
NF601	프로젝트 관리	Git 활용	개발자 간 작업 충돌이 일어나지 않게끔 Git을 활용하여 프로젝트 관리	필수

NF602	유지보수	URL 구성	URL이 변경될 때 유지보수가 용이하도록 구현	필수
NF603	보안	HTTP Method 허용	허용된 HTTP Method를 사용하는 요청만 허락하도록 구현	필수

1. 프로젝트 관리 (필수)

1) 프로젝트 관리

① 요구사항 번호 : NF601

② Git 의 기능을 적극적으로 활용하여, 여러 개발자가 동시에 개발할 수 있는 환경을 조성합니다.

- 팀원과 합의하여 branch 생성 원칙 정리하기
  - ◆ Gitflow, GitHub Flow 등을 참고하여 원칙을 정할 수 있음
  - ◆ branch 운영 중 문제상황이 발생할 경우 원인에 대한 분석을 꼭 진행할 것
- 팀원과 합의하여 commit 내역 기록 원칙 정리하기
  - ◆ 기능 개발, 버그 수정 등 그 목적이 명확히 드러날 수 있도록 commit 메시지를 작성할 것
  - ◆ 어느 시점에 commit 을 남길지에 대하여 충분한 합의를 이룰 것
- 프로젝트 종료 시 초기에 세운 원칙이 잘 지켜졌는지 점검
  - ◆ README 에 초기에 세운 원칙을 설명하여 정리할 것
  - ◆ 생성했던 branch 들에 대한 용도를 정리할 것
  - ◆ 작성된 commit 들을 GitLab 에서 확인하여 스크린샷으로 README 에 포함시킬 것

2. 프로젝트 및 앱

도서 데이터를 생성, 조회, 수정, 삭제할 수 있는 Django 프로젝트를 만듭니다. 명시된 요구사항 이외 서비스를 위해 필요하다고 생각되는 기능 등은 자유롭게 구현해도 무관하며, Bootstrap 을 활용하여 자유롭게 스타일링 합니다.

1) 기본 빌드

① 요구사항 번호 : F601, NF602, NF603

- 프로젝트 이름 : mypjt
- 앱 이름 : books
- Model 클래스 이름 : books, accounts
- 페이지간 이동 편의성을 갖추기 위한 Navbar 구성
  - ◆ books 의 index, create 와 accounts 의 기능들에 접근하기 편하도록 링크를 제공
  - ◆ 다양한 페이지에서 도달할 수 있도록 여러 페이지에 동일하게 출력되도록 구현

2) accounts

① 요구사항 번호 : F602, F603

② 사용자 데이터를 관리하기 위한 앱

- Model 클래스
  - ◆ Django 에서 제공하는 AbstractUser 를 상속받는 커스텀 모델 클래스 구현
  - ◆ 아이디와 비밀번호 외에 이메일, 성, 이름, 성별, 나이, 주간 평균 독서 시간, 연간 독서량, 프로필 사진을 저장할 수 있어야 함.
  - ◆ 팔로우 기능을 위해 User 는 다른 User 와 M:N 관계를 맺어야 함.
- Form 클래스

◆ 상황에 따라 Django가 제공하는 Form과 필요한 경우 커스텀 Form 클래스를 적절히 활용

- 각 세부 기능들을 위한 view 함수를 구성
  - ◆ login, logout, signup, profile, update, delete, change_password
  - ◆ URL이 변경될 가능성을 고려하여 유지보수에 유리한 형태로 구성
  - ◆ 각 view 함수가 허용하는 HTTP Method에만 호출되도록 구현

3) books

① 요구사항 번호 : F604, F605, F606

② 도서 데이터를 관리하기 위한 앱

- Model 클래스
  - ◆ Book: 도서 데이터를 나타내는 Model. 도서 제목, 설명, 회원 리뷰 평점, 저자를 저장할 필드 4개 지정
  - ◆ Thread: 도서에 대한 쓰레드(게시글)을 나타내는 Model. 쓰레드 제목, 본문, 독서일, 생성 시각, 수정 시각을 저장할 필드 5개 지정
  - ◆ 쓰레드를 나타내는 Thread는 Book과 N:1 관계를 맺고 있어야 함.
  - ◆ Book, Thread는 각각 작성자를 나타내는 User 클래스와 1:N 관계를 맺고 있어야 함.
  - ◆ 좋아요 기능을 위해 Thread와 User는 M:N 관계를 맺어야 함.
- Form 클래스
  - ◆ 각 Model 클래스에 대하여 데이터 검증, 저장, 에러메시지 등을 관리하기 위해 적절한 ModelForm 사용

- 각 세부 기능들을 위한 view 함수를 구성
  - ◆ index, create, detail, update, delete, thread_create, thread_detail, thread_update, thread_delete
  - ◆ URL 이 변경될 가능성을 고려하여 유지보수에 유리한 형태로 구성
  - ◆ 각 view 함수가 허용하는 HTTP Method 에만 호출되도록 구현

3. Accounts 앱 view 함수

사용자 데이터를 관리하는 역할을 하는 accounts 앱의 view 함수를, Django Authentication System 을 활용하여 구현해봅시다.

1) Login

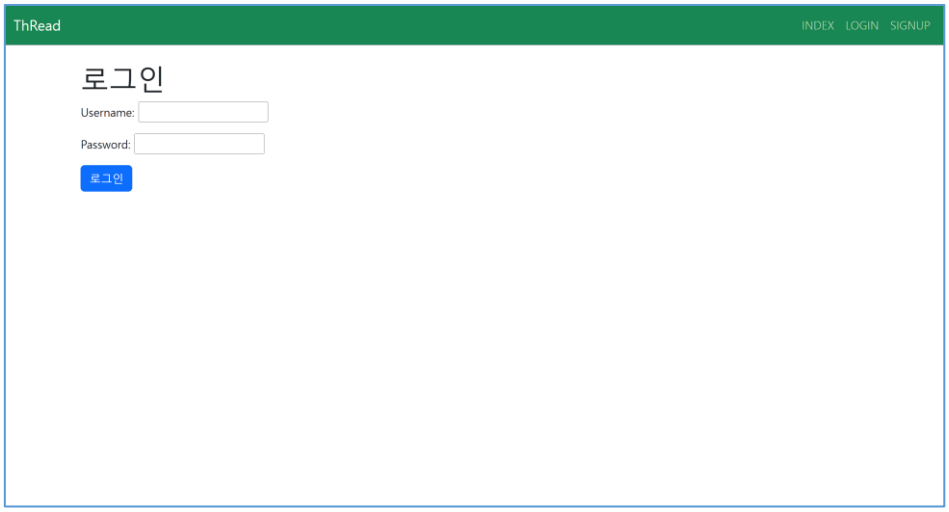
① 요구사항 번호 : F607

② 사용자가 자기 정보를 입력할 수 있는 UI 를 제공하고, 로그인할 수 있도록 해주는 login view 함수를 구현

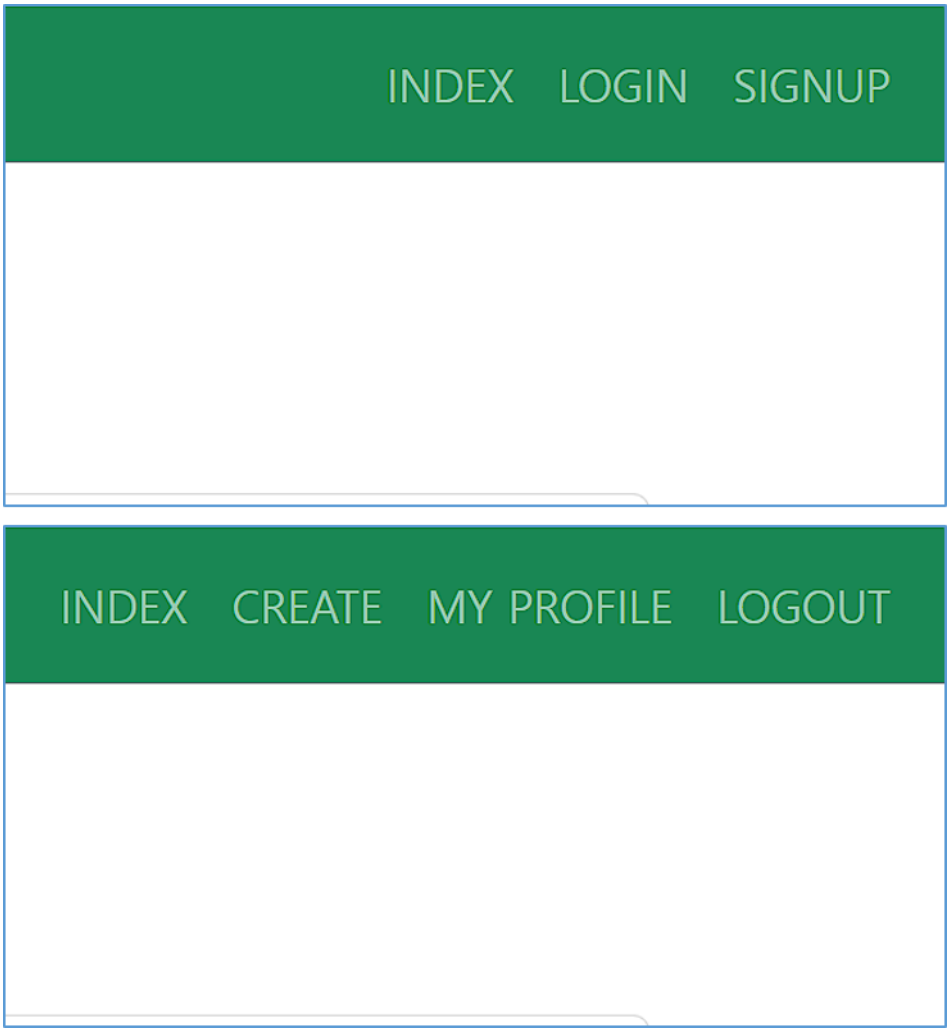
- 사용자가 로그인 되어 있지 않을 때 정상적으로 동작
  - ◆ 로그인 되어 있지 않은 상태에서만 출력되는 로그인 링크를 통해 이동
  - ◆ 사용자가 제공한 데이터의 유효성 검증이 이뤄져야 함
  - ◆ 로그인에 성공할 경우 메인 페이지로 redirect
- 사용자의 로그인 여부에 따라 사이트 UI 가 변화되어야 함
  - ◆ 로그인 링크가 생성, 나의 프로필, 로그아웃 링크 등으로 변경
  - ◆ 로그인이 되어있지 않은 경우 데이터 생성 링크 출력되지 않음
- 같은 view 함수에서 HTML 제공과 로그인이 같이 이뤄지도록 구현

◆ GET 요청을 받을 경우: HTML 제공

◆ POST 요청을 받을 경우: 유효한 사용자 데이터의 경우 세션 정보 생성(로그인), 아닌 경우 오류를 페이지에 표시



<그림 1> 요구사항 F607 결과물 예시



<그림 2> 요구사항 F607 결과물 예시 - 로그인 전후 링크 변경

2) Logout

① 요구사항 번호 : F608

② 사용자가 로그아웃 할 수 있는 UI 를 제공하고, 로그아웃을 해주는 logout view 함수를 구현

- 사용자가 로그인 되어 있을 때 정상적으로 동작

◆ 로그인 된 상태에서만 출력되는 로그아웃 UI 를 통해 사용

◆ 로그아웃에 성공할 경우, 메인 페이지로 redirect

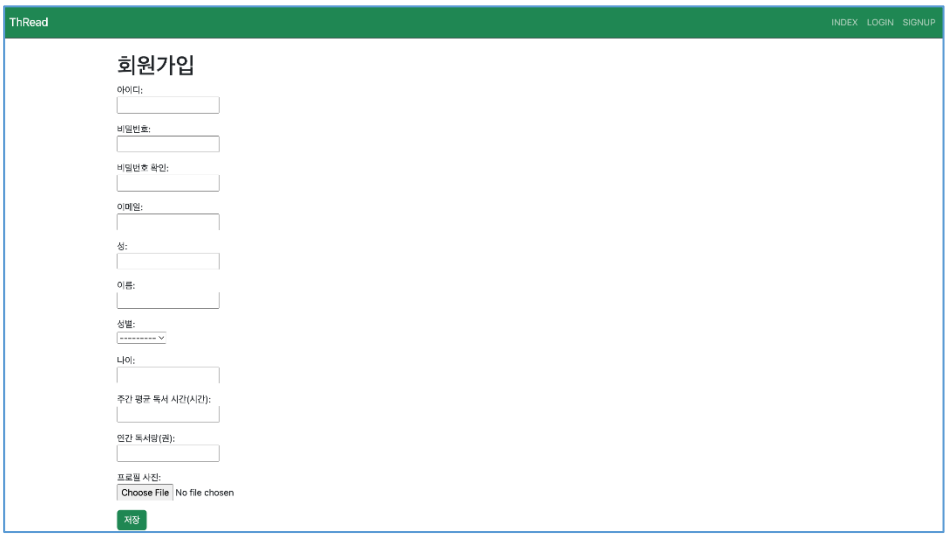
- POST 요청에 대해서만 작동하도록 구현

3) signup

① 요구사항 번호 : F609

② 아직 등록되지 않은 사용자가 데이터를 입력할 수 있는 UI 를 제공하고, 회원가입을 진행할 수 있도록 해주는 signup view 함수를 구현

- 사용자가 로그인 되어 있지 않을 때 정상적으로 동작
  - ◆ 로그인 되어 있지 않은 상태에서만 출력되는 회원가입 링크를 통해 이동
  - ◆ 사용자가 제공한 데이터의 유효성 검증이 이뤄져야 함
  - ◆ 회원가입 완료시 로그인을 지냉ㅇ하고 메인 페이지로 redirect
- 같은 view 함수에서 HTML 제공과 회원가입이 같이 이뤄지도록 구현
  - ◆ GET 요청을 받을 경우: HTML 제공
  - ◆ POST 요청을 받을 경우: 유효한 데이터의 경우 새로운 사용자 정보 생성, 세션 정보 생성(로그인), 아닌 경우 오류를 페이지에 표시



<그림 3> 요구사항 F609 결과물 예시

4) profile

① 요구사항 번호 : F610

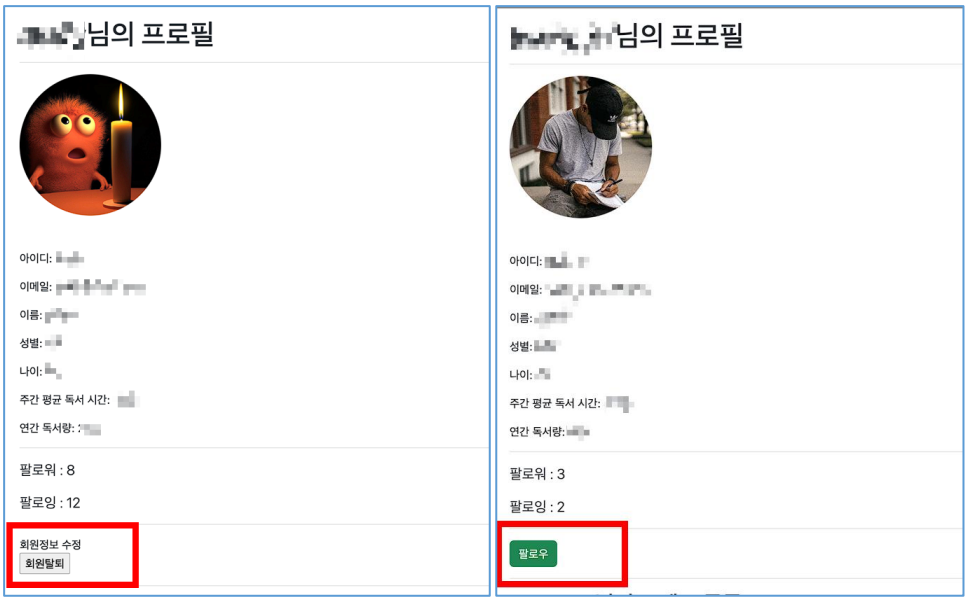
② 개별 사용자의 정보를 확인할 수 있는 페이지 구현

- 사용자의 로그인 여부와 상관 없이 동작

◆ 대상 사용자의 회원 데이터, 작성한 쓰레드, 팔로우 수, 팔로잉 수 확인 가능

◆ 인증된 사용자가 자신과 다른 사용자의 프로필 페이지를 확인하는 경우, 해당 사용자를 팔로우 하는지 여부에 따라 팔로우 / 언팔로우 버튼 표시

◆ 현재 인증된 사용자와 동일한 사용자의 프로필 페이지인 경우, 회원정보수정, 회원탈퇴 링크를 표시



<그림 4> 요구사항 F610 결과물 예시

5) follow

① 요구사항 번호 : F611

② 한 사용자가 다른 사용자를 팔로우 하는 기능

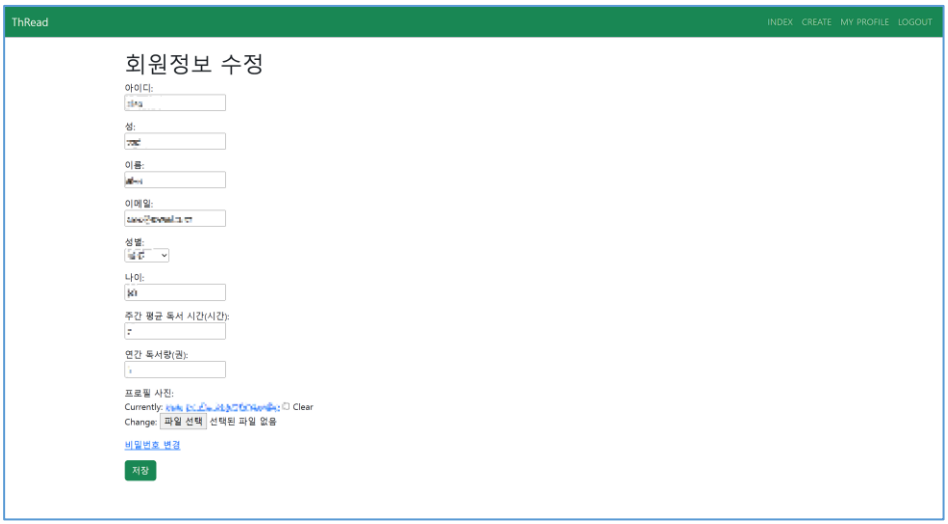
- 사용자가 로그인 되어 있을 때 정상적으로 동작
  - ◆ 로그인 된 상태에서 인증된 사용자가 자신과 다른 사용자의 프로필 페이지에서 확인할 수 있는 팔로우 / 언팔로우 버튼을 통해 사용
  - ◆ 팔로우 중인 상태에선 언팔로우, 팔로우 중이지 않은 상태에선 팔로우가 되게끔 동작
  - ◆ 성공할 경우 현재 페이지로 redirect
- POST 요청에 대해서만 작동하도록 구현

6) update

③ 요구사항 번호 : F612

④ 사용자가 자신의 정보를 입력할 수 있는 UI 를 제공하고, 회원 정보를 수정할 수 있도록 해주는 update view 함수를 구현

- 사용자가 로그인 되어 있을 때 정상적으로 동작
  - ◆ 로그인 된 상태에서만 출력되는 회원정보수정 링크를 통해 이동
  - ◆ 회원정보수정에 성공할 경우 메인 페이지로 redirect
- 같은 view 함수에서 HTML 제공과 회원정보수정이 같이 이뤄지도록 구현
  - ◆ GET 요청을 받을 경우: HTML 제공, 단 일반적인 사용자가 접근하면 안 되는 정보는 출력되지 않아야 함
  - ◆ POST 요청을 받을 경우: 유효한 데이터의 경우 로그인 된 사용자 정보 수정, 아닌 경우 오류를 페이지에 표시



<그림 5> 요구사항 F612 결과물 예시

7) delete

① 요구사항 번호 : F613

② 사용자가 회원탈퇴를 할 수 있는 UI 를 제공하고, 회원탈퇴를 해주는 delete view 함수를 구현

- 사용자가 로그인 되어 있을 때 정상적으로 동작
  - ◆ 로그인 된 상태에서만 출력되는 회원탈퇴 UI 를 통해 사용
  - ◆ 회원탈퇴에 성공할 경우 메인 페이지로 redirect
- POST 요청에 대해서만 작동하도록 구현

8) change_password

① 요구사항 번호 : F614

② 사용자가 자신의 비밀번호를 수정할 수 있도록 비밀번호를 입력하는 UI 를 제공하고, 비밀번호를 수정해주는 change_password view 함수를 구현

- 사용자가 로그인 되어 있을 때 정상적으로 동작

◆ 로그인 된 상태에서만 출력되는 비밀번호수정 UI, 또는 회원정보수정 페이지에 Django 가 기본적으로 제공하는 링크를 통해 이동

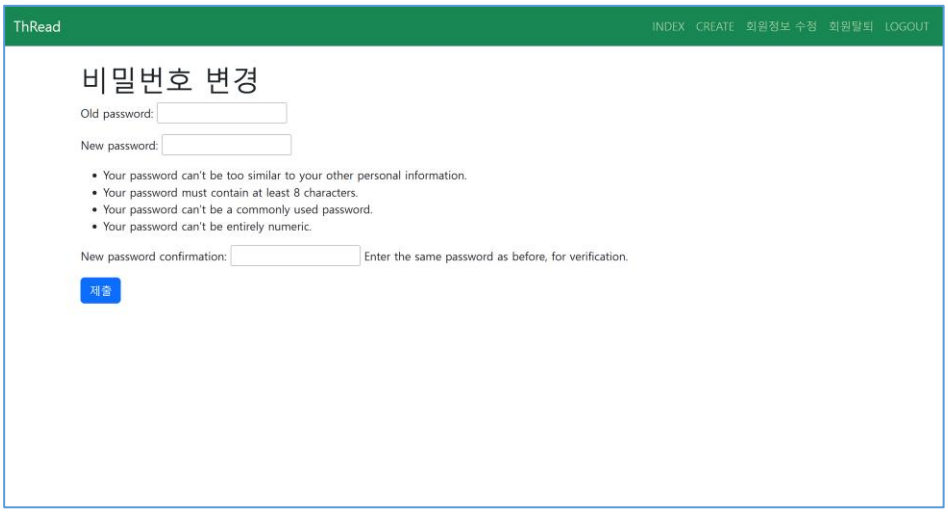
◆ 비밀번호 수정에 성공할 경우 메인 페이지로 redirect

◆ 비밀번호 수정 후 강제로 로그아웃 되지 않도록 구현

- 같은 view 함수에서 HTML 제공과 비밀번호변경이 같이 이뤄지도록 구현

◆ GET 요청을 받을 경우: HTML 제공

◆ POST 요청을 받을 경우: 유효한 데이터의 경우 로그인 된 사용자의 비밀번호 수정, 아닌 경우 오류를 페이지에 표시



<그림 6> 요구사항 F614 결과물 예시

4. Books 앱 view 함수

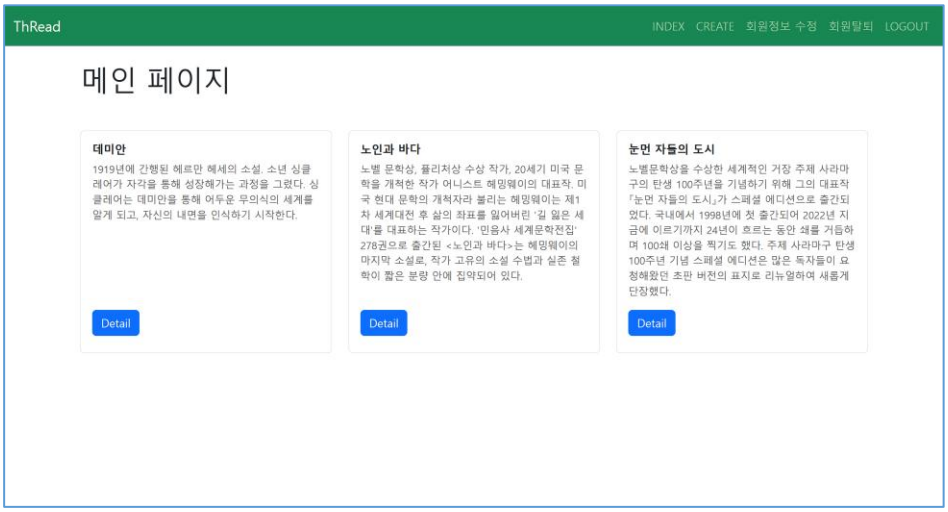
도서 데이터를 생성, 조회, 수정, 삭제하는 역할을 하는 books 앱을 구현해봅시다..

1) index

① 요구사항 번호 : F615

② 전체 도서 데이터를 조회하기 위한 index view 함수 구현

- 전체 도서 데이터가 조회되어야 함
- 특정 도서를 선택하여 detail 페이지로 이동할 수 있는 링크가 제공되어야 함
- 링크의 문구와 UI 는 자유롭게 구성



<그림 7> 요구사항 F615 결과물 예시

2) create

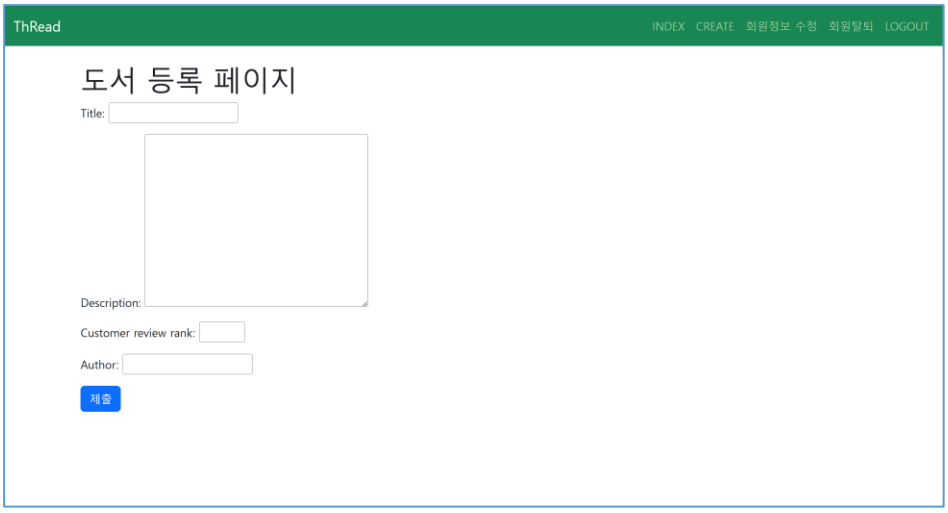
① 요구사항 번호 : F616

② 도서 데이터를 입력하기 위한 UI 를 제공하고, 입력된 데이터를 저장하는 create view 함수를 구현

- Book 모델의 데이터를 입력할 수 있는 UI 구현
  - ◆ 유효하지 않은 데이터가 저장되지 않도록 구현
  - ◆ 현재 로그인한 사용자와 1:N 관계를 맺도록 구현

- ◆ 로그인 되어있지 않은 경우 도서 작성 불가
- ◆ 유효한 데이터의 경우 저장 후 작성된 도서의 detail 페이지로 redirect
- ◆ UI 디자인은 자유롭게 구성

- 같은 view 함수에서 HTML 제공과 데이터 생성이 같이 이뤄지도록 구현
  - ◆ GET 요청을 받을 경우: HTML 제공
  - ◆ POST 요청을 받을 경우: 형식에 맞는 데이터는 저장, 형식에 맞지 않는 데이터는 오류를 페이지에 표시



<그림 8> 요구사항 F616 결과물 예시

3) detail

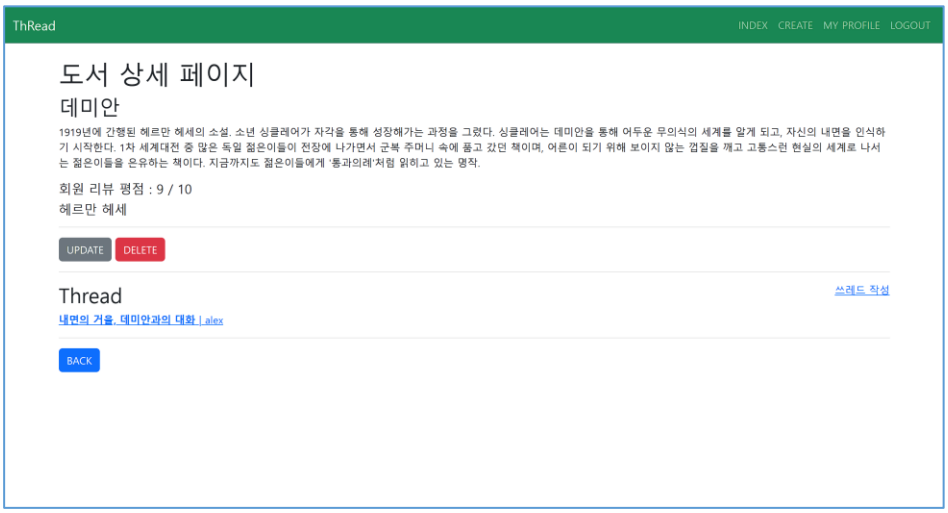
① 요구사항 번호 : F617

② 단일 도서 데이터를 조회하는 detail view 함수 구현

- index 페이지에서 도서를 선택했을 때 이동됨

◆ 선택한 해당 도서의 정보를 바탕으로 내용이 출력되어야 함

- 도서 제목, 설명, 회원 리뷰 평점, 저자 데이터를 출력
  - ◆ 도서를 수정하기 위한 페이지로 이동하는 링크 구현
  - ◆ 도서를 삭제하는 버튼 구현
  - ◆ 수정 링크 및 삭제 버튼은 도서를 작성한 사용자만 확인 및 사용 가능
  - ◆ 이전 페이지(index)로 이동하는 링크 구현
  - ◆ UI 디자인은 자유롭게 구성
- 쓰레드 목록 및 작성 UI 출력
  - ◆ 쓰레드 제목이 목록으로 보이도록 구현
  - ◆ 쓰레드를 작성할 수 있는 페이지로 이동하는 링크 구현



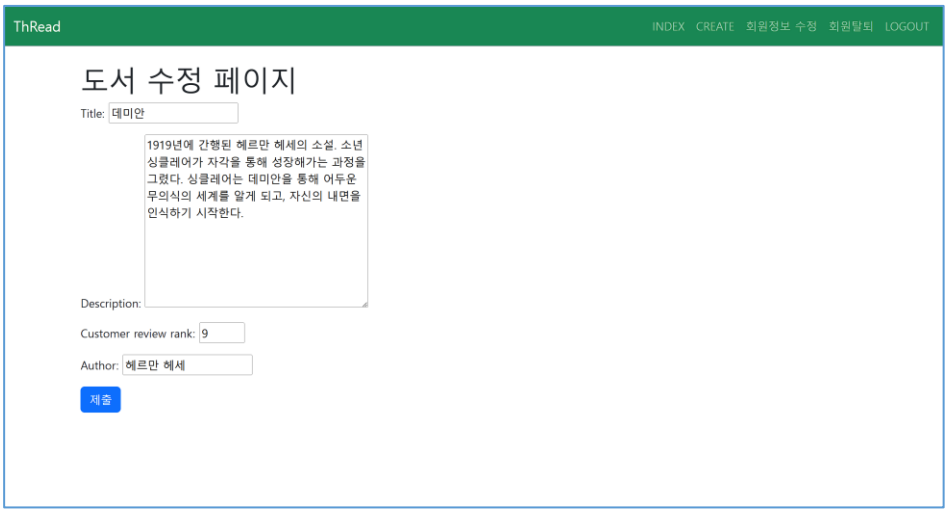
<그림 9> 요구사항 F617 결과물 예시

4) update

① 요구사항 번호 : F618

② 도서 데이터를 수정하기 위한 UI를 제공하고, 입력된 데이터를 바탕으로 대상 도서 데이터를 수정하는 update view 함수를 구현

- detail 페이지에서 수정하기 링크를 통해 이동됨
  - ◆ detail 페이지에서 제공되었던 도서 정보를 바탕으로 기능이 동작되도록 구현
  - ◆ 도서를 작성한 사용자만 수정할 수 있도록 구현
- Book 모델의 데이터를 입력할 수 있는 UI 구현
  - ◆ 이전 도서의 데이터가 미리 입력되어 있도록 구현
  - ◆ 유효하지 않은 데이터가 저장되지 않도록 구현
  - ◆ 유효한 데이터의 경우 저장 후 작성된 도서의 detail 페이지로 redirect
  - ◆ UI 디자인은 자유롭게 구성
- 같은 view 함수에서 HTML 제공과 데이터 수정이 같이 이뤄지도록 구현
  - ◆ GET 요청을 받을 경우: HTML 제공
  - ◆ POST 요청을 받을 경우: 형식에 맞는 데이터는 저장, 형식에 맞지 않는 데이터는 오류를 페이지에 표시



<그림 10> 요구사항 F618 결과물 예시

5) delete

① 요구사항 번호 : F619

② 도서 데이터를 삭제하는 기능을 제공하는 delete view 함수를 구현

- detail 페이지에서 삭제하기 버튼을 통해 호출됨

◆ detail 페이지에서 제공되었던 도서 정보를 바탕으로 기능이 동작되도록 구현

◆ 도서를 작성한 사용자만 삭제할 수 있도록 구현

◆ POST 요청에 대해서만 작동하도록 구현

◆ 삭제가 된 이후에는 메인 페이지로 redirect

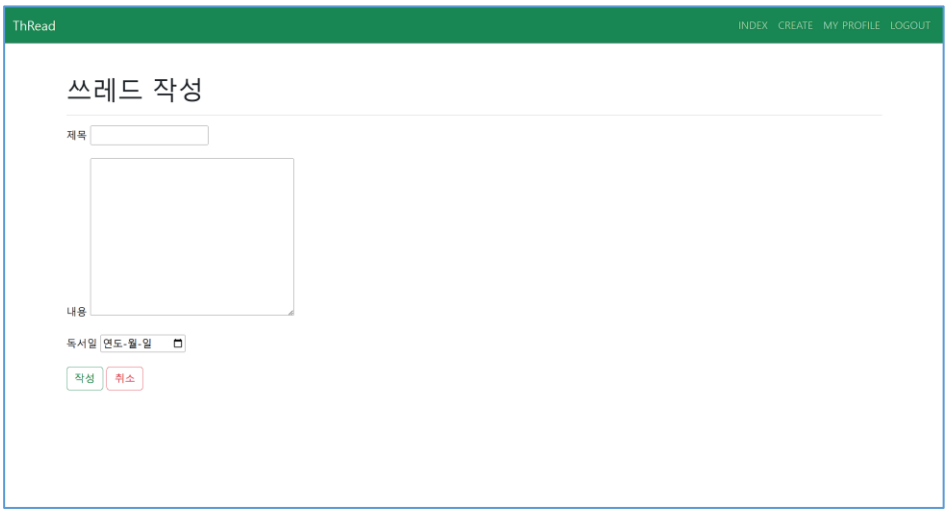
6) thread_create

① 요구사항 번호 : F620

② 쓰레드 데이터를 입력하기 위한 UI 를 제공하고, 입력된 데이터를 저장하는

thread_create view 함수를 구현

- Thread 모델의 데이터를 입력할 수 있는 UI 구현
  - ◆ 유효하지 않은 데이터가 저장되지 않도록 구현
  - ◆ 현재 로그인한 사용자와 1:N 관계를 맺도록 구현
  - ◆ 로그인 되어있지 않은 경우 쓰레드 작성 불가
  - ◆ 유효한 데이터의 경우 저장 후 작성된 쓰레드 detail 페이지로 redirect
  - ◆ UI 디자인은 자유롭게 구성
- 같은 view 함수에서 HTML 제공과 데이터 생성이 같이 이뤄지도록 구현
  - ◆ GET 요청을 받을 경우: HTML 제공
  - ◆ POST 요청을 받을 경우: 형식에 맞는 데이터는 저장, 형식에 맞지 않은 데이터는 오류를 페이지에 표시



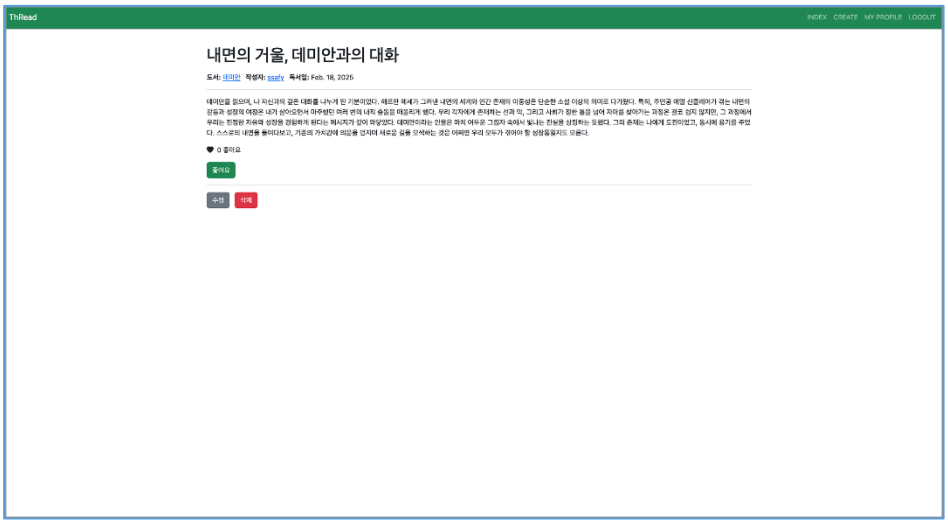
<그림 11> 요구사항 F620 결과물 예시

7) thread_detail

① 요구사항 번호 : F621

② 단일 쓰레드 데이터를 조회하는 thread_detail view 함수 구현

- detail 페이지에서 쓰레드를 선택했을 때 이동됨
  - ◆ 선택한 쓰레드의 정보를 바탕으로 내용이 출력되어야 함
- 쓰레드 제목, 본문, 독서일 데이터를 출력
  - ◆ 쓰레드를 수정하기 위한 페이지로 이동하는 링크 구현
  - ◆ 쓰레드를 삭제하는 버튼 구현
  - ◆ 수정 링크 및 삭제 버튼은 쓰레드를 작성한 사용자만 확인 및 사용 가능
  - ◆ 이전 페이지(detail)로 이동하는 링크 구현
  - ◆ 쓰레드를 좋아요 하는 버튼 구현
  - ◆ UI 디자인은 자유롭게 구성



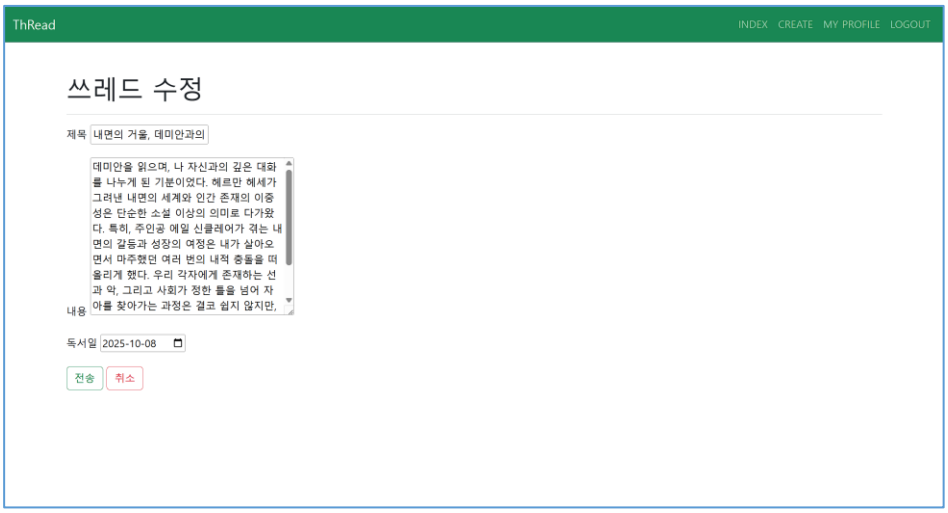
<그림 12> 요구사항 F621 결과물 예시

8) thread_update

① 요구사항 번호 : F622

② 쓰레드 데이터를 수정하기 위한 UI 를 제공하고, 입력된 데이터를 바탕으로 대상 쓰레드 데이터를 수정하는 thread_update view 함수를 구현

- thread_detail 페이지에서 수정하기 링크를 통해 이동됨
  - ◆ thread_detail 페이지에서 제공되었던 쓰레드 정보를 바탕으로 기능이 동작되도록 구현
  - ◆ 쓰레드를 작성한 사용자만 수정할 수 있도록 구현
- Thread 모델의 데이터를 입력할 수 있는 UI 구현
  - ◆ 이전 쓰레드의 데이터가 미리 입력되어 있도록 구현
  - ◆ 유효하지 않은 데이터가 저장되지 않도록 구현
  - ◆ 유효한 데이터의 경우 저장 후 수정된 쓰레드의 thread_detail 페이지로 redirect
  - ◆ UI 디자인은 자유롭게 구성
- 같은 view 함수에서 HTML 제공과 데이터 수정이 같이 이뤄지도록 구현
  - ◆ GET 요청을 받을 경우: HTML 제공
  - ◆ POST 요청을 받을 경우: 형식에 맞는 데이터는 저장, 형식에 맞지 않는 데이터는 오류를 페이지에 표시



<그림 13> 요구사항 F622 결과물 예시

9) thread_delete

① 요구사항 번호 : F623

② 쓰레드 데이터를 삭제하는 기능을 제공하는 thread_delete view 함수를 구현

- thread_detail 페이지에서 삭제하기 버튼을 통해 호출됨
  - ◆ thread_detail 페이지에서 제공되었던 쓰레드 정보를 바탕으로 기능이 동작되도록 구현
  - ◆ 쓰레드를 작성한 사용자만 삭제할 수 있도록 구현
  - ◆ POST 요청에 대해서만 작동하도록 구현
  - ◆ 삭제가 된 이후에는 관계를 맺고 있는 도서 데이터의 detail 페이지로 redirect

10) like

③ 요구사항 번호 : F624

④ 쓰레드 데이터를 좋아요 하거나 취소하는 like view 함수를 구현

- thread_detail 페이지에서 좋아요 버튼을 통해 호출됨

- ◆ thread_detail 페이지에서 제공되었던 쓰레드 정보를 바탕으로 기능이 동작되도록 구현
- ◆ 쓰레드를 좋아요 한 상태에선 좋아요 취소, 좋아요 중이지 않은 상태에선 좋아요 하게끔 동작
- ◆ 성공할 경우 현재 페이지로 redirect

- POST 요청에 대해서만 작동하도록 구현

2. 심화 과제 (선택)

생성형 AI 도구를 활용하여 심화 과제 요구사항을 해결해봅시다. 코드 생성, 아이디어 구상, 문제 해결 방법 탐색 등 다양한 방식으로 활용할 수 있습니다. 사용할 생성형 AI 서비스는 자유롭게 선택하고, 최종 결과물은 AI 생성 내용을 바탕으로 직접 수정 및 개선하여 적용해봅시다.

생성형 AI 는 보조 수단으로 활용하되, 능동적인 자세로 학습에 임합니다. 최종적인 이해와 적용은 자기 주도적 학습을 통해 이루어지며, 배운 내용을 스스로 기록하고 정리하며 학습 효과를 높여 봅시다.

1) 생성형 AI API 를 활용한 작가 정보 생성 및 저장

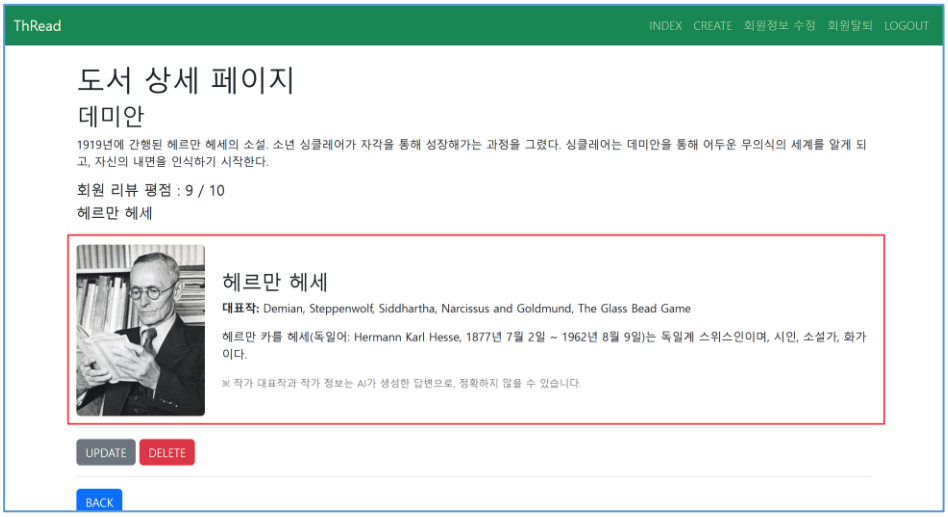
① 요구사항 번호 : F625

② 생성형 AI API 를 활용하여 작가 정보를 생성하고 저장

- 도서를 등록할 때 BookForm 으로 입력받은 작가 이름을 위키피디아 API 의 쿼리로 전달해 작가 정보, 프로필 사진 검색
- 사용자 입력 데이터와 위키피디아 검색 정보를 바탕으로, GPT API 를 활용하기

- ◆ 제공되는 GPT API Key를 활용할 경우, 반드시 gpt-5-nano 모델을 사용
- ◆ 작가 정보와 대표 작품 목록 요청
- ◆ JSON 형태로 응답하도록 프롬프트 작성

- Book 모델에 필드를 추가하여 저장
  - ◆ author_info와 author_works 필드를 추가하여 저장
- 추가한 필드를 도서 detail 페이지에 출력
  - ◆ 작가 정보 영역 아래에 "작가 대표작과 작가 정보는 AI가 생성한 정보로, 정확하지 않을 수 있습니다."라는 안내 문구 추가



<그림 14> 요구사항 F625 결과물 예시

2) 생성형 AI를 활용한 쓰레드 커버 이미지 생성

① 요구사항 번호 : F626

② 쓰레드 작성시 커버 이미지를 생성형 AI로 생성

- 도서 제목, 저자 이름, 독서 쓰레드의 감정을 분석해 키워드 5 개를 추출
  - ◆ 추출을 위해 생성형 AI 활용
  - ◆ 제공되는 GPT API Key 를 활용할 경우, 반드시 gpt-5-nano 모델을 사용
- 추출한 키워드를 기반으로 이미지 생성을 위한 프롬프트 작성
  - ◆ 작성된 프롬프트를 통해 어떤 이미지를 그려야 할지 결정 후 생성형 AI 에게 전달
  - ◆ 제공되는 GPT API Key 를 활용할 경우, 반드시 dall-e-3 모델을 사용
- 생성된 이미지를 파일 시스템에 저장 후 Thread 모델에 필드를 추가하여 저장
  - ◆ cover_image 필드를 추가하여 저장



<그림 15> 요구사항 F626 결과물 예시

3) 쓰레드 댓글

① 요구사항 번호 : F627

② 쓰레드에 대한 댓글 생성 및 삭제 기능 구현

- 쓰레드 상세보기 페이지에서 사용 가능
  - ◆ 기능 구현을 위한 적절한 Django Model 클래스 추가
  - ◆ 쓰레드 상세보기 페이지에 댓글 목록, 댓글 작성 UI 추가
  - ◆ 유효하지 않은 데이터가 저장되지 않도록 구현
  - ◆ 현재 로그인한 사용자와 1:N 관계를 맺도록 구현
  - ◆ 로그인 되어있지 않은 경우 쓰레드 댓글 작성 불가
  - ◆ 댓글 목록의 댓글 옆에 작성한 사용자의 프로필 페이지로 이동 가능한 링크 구현
  - ◆ 댓글 목록의 댓글 중 현재 로그인한 사용자와 동일한 사용자만 확인 및 사용 가능한 삭제 버튼 구현



<그림 16> 요구사항 F627 결과물 예시



<그림 1> 요구사항 F627 결과물 예시

4) 카테고리

① 요구사항 번호 : F628

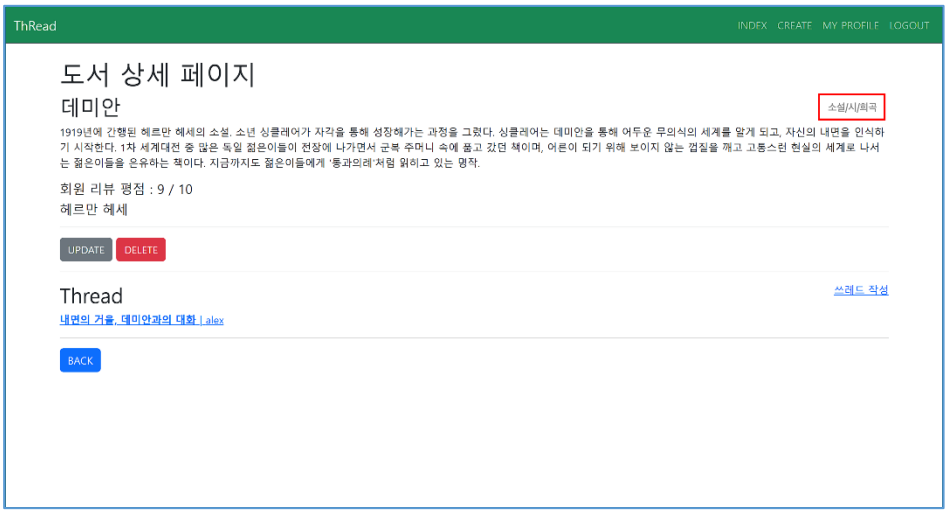
② 사용자의 선호 장르와 도서의 장르를 나타내기 위한 카테고리 Django Model 구현

- 장르 목록
  - ◆ 소설/시/희곡, 경제/경영, 자기계발, 인문/교양, 취미/실용, 어린이/청소년, 과학
- 카테고리와 사용자 모델은 M:N 관계를 맺음
  - ◆ 회원정보 입력, 수정 시 선호하는 카테고리를 선택 가능하도록 구현
  - ◆ 다중 선택 가능하도록 구현
  - ◆ 프로필 페이지에 선호 카테고리 출력
- 카테고리와 도서 모델은 1:N 관계를 맺음
  - ◆ 도서 등록 시 카테고리 선택
  - ◆ 도서 목록 페이지에서 카테고리를 바탕으로 도서 필터링 기능 추가

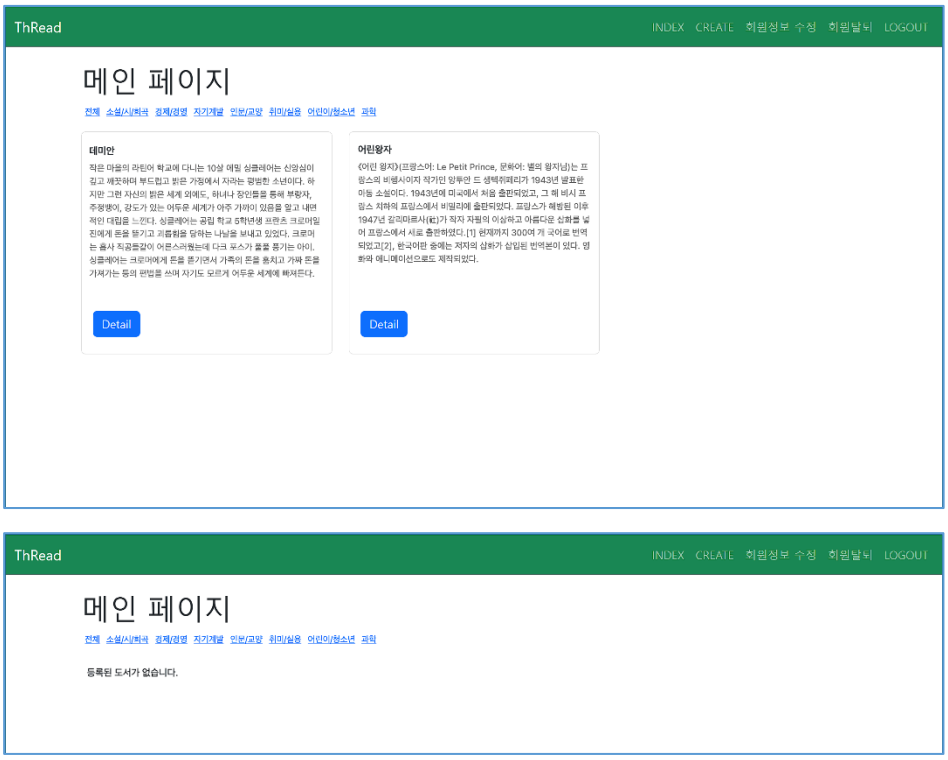
對外秘



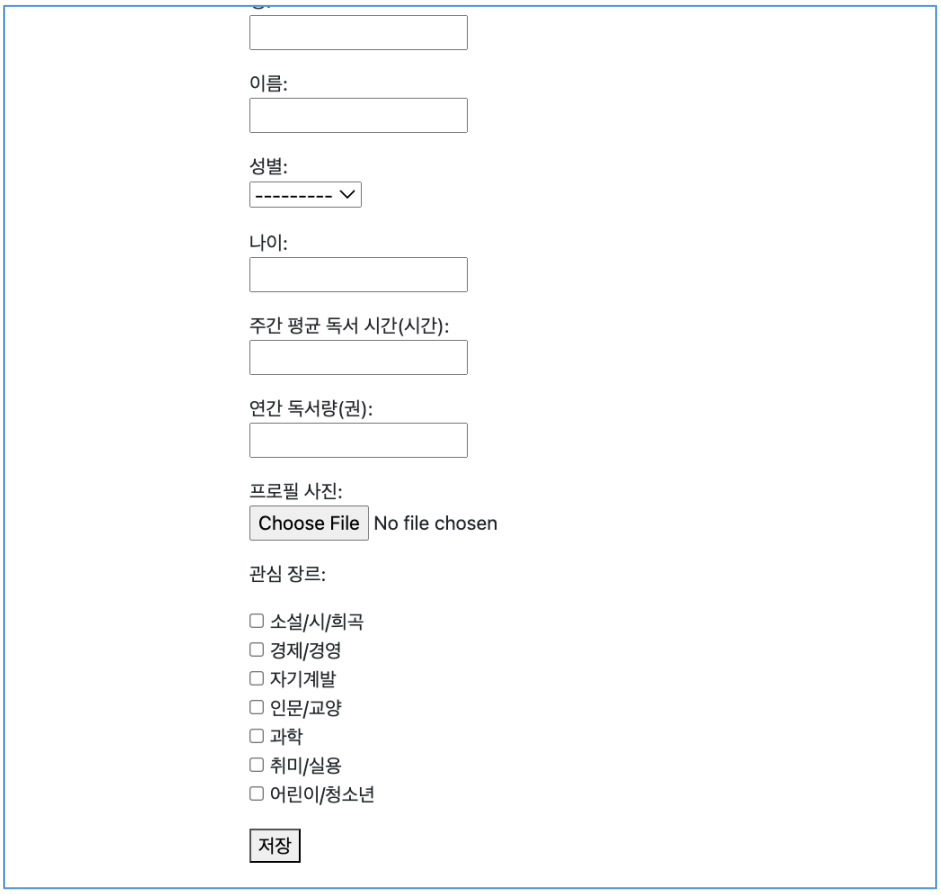
<그림 18> 요구사항 F628 결과물 예시 - 도서 등록 페이지



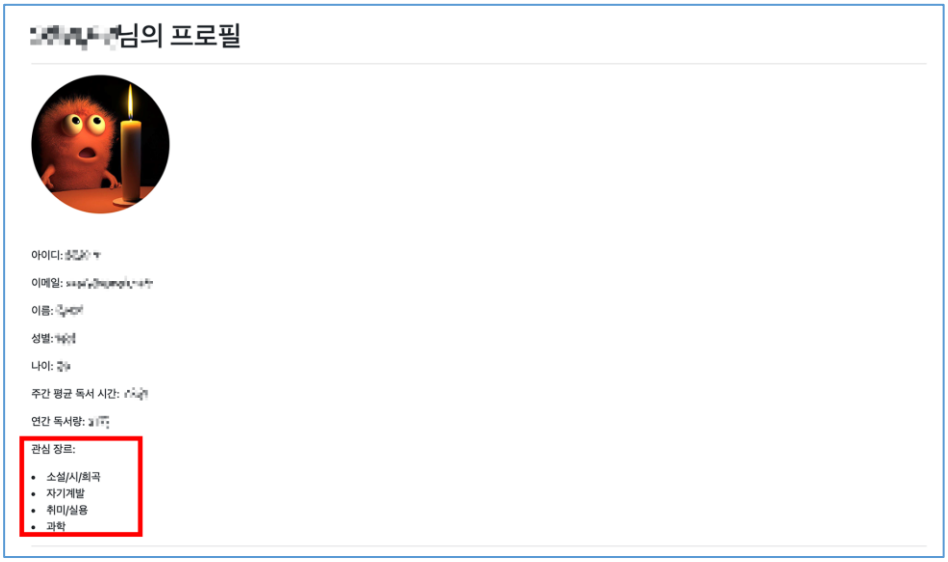
<그림 19> 요구사항 F628 결과물 예시 - 도서 상세 페이지



<그림 20> 요구사항 F628 결과물 예시 - 메인 페이지 필터 적용



<그림 21> 요구사항 F628 결과물 예시 - 회원가입 페이지



<그림 22> 요구사항 F628 결과물 예시 - 프로필 페이지

1.5 참고자료

- Django 5.2
  https://www.djangoproject.com/start/overview/

- Python OpenAI Library
  https://platform.openai.com/docs/libraries?language=python
  https://github.com/openai/openai-python

- OpenAI 이미지 생성 API 문서
  https://platform.openai.com/docs/guides/image-generation

1.6 프로젝트 결과

아래 내용들을 모두 포함하여 GitLab 에 업로드한다.

프로젝트 이름은 프로젝트 번호 + -pjt 로 지정한다. (ex) 06-pjt)

■ **산출물과 제출**

1) 구현 소스 코드

◆ 프로젝트 전체 소스 코드를 제출한다.

2) 실행 결과 캡처본

◆ 각 요구사항에 대한 실행 결과 화면을 캡처한다.

3) README.md

◆ 주요 기능 설명, 학습 내용, 느낀 점을 반드시 포함한다.

- 以上